SO-WebServer: Design Document

1. Introduction

The purpose of this document is to present the architectural design of the *ConcurrentHTTP Server* system, a multi-process and multi-threaded web server developed using POSIX and Unix Domain Sockets (UDS) inter-process communication mechanisms.

This design document describes the structure, behaviour, and interaction of all major components of the system, providing a clear understanding of how client requests are received, dispatched, processed, and returned.

The server was developed with the intent to support multiple concurrent connections while assuring data's integrity. This was achieved by separating connection handling (master) from request processing (worker), distributing work across multiple processes (workers) and multiple threads within each worker.

As referred above, inter-process communication was accomplished by having a shared memory that different processes can access, using POSIX semaphores to avoid race conditions and ensure data integrity and consistency. We also used UDS to delegate accepted connections to the worker processes. Further details on this system's implementation will be available on the Technical Report.

The following sections present the architectural design of the system, the internal behaviour of each component, and the synchronization strategies that enable reliable concurrent execution.

2. System Architecture

2.1. Overview

This section provides a conceptual view of the system's architecture, focusing on how the major components are organised and how they interact at a structural level. Rather than covering implementation details, the goal is to describe the overall model the server follows, the roles of the master and worker entities, and the general flow of requests through the system.

Key principles of this system:

- **Separation of concerns:** The master focuses on connection acceptance and safe hand-off of work; workers focus on executing request handling logic.
- **Coordination:** Inter-process coordination is achieved through a shared memory that is controlled with POSIX semaphores.
- **Resilience considerations:** Failures in a single worker should not compromise the master or other workers. The master can continue accepting new connections while individual workers are restarted or replaced, enabling graceful degradation and easier recovery strategies.

2.2. Main Components

2.2.1 Master Process

The master process is responsible for managing all incoming client connections. It listens for new TCP connections and performs the initial acceptance step. Instead of processing requests directly, the master delegates the handling of each accepted connection to the worker layer. This design minimises bottlenecks by ensuring that the master remains dedicated to connection intake, allowing the system to scale with higher incoming traffic rates.

2.2.2 Worker Processes

Worker processes handle the actual HTTP request processing. They operate independently from one another, enabling parallel execution. Each worker receives client connections from the master (using UDS) and coordinates their processing through its internal thread pool. Workers focus exclusively on request handling, which reduces complexity and promotes modularity within the system.

2.2.3 Thread Pool

Each worker process contains a fixed-size pool of worker threads. These threads are responsible for executing the request-handling tasks associated with each client connection. The thread pool model improves responsiveness and enables the worker to process multiple requests concurrently.

2.2.4 Inter-Process Communication Layer

The system relies on three complementary IPC mechanisms to coordinate work between components:

Shared Memory Region

A shared memory is used as a communication structure to update server statistics. The region is accessible to both the master and all workers.

POSIX Semaphores

Semaphores ensure mutual exclusion and proper synchronization when processes access shared structures. They prevent race conditions and ensure that shared data remains consistent even when multiple processes operate concurrently.

Unix Domain Sockets (UDS)

UDS provides a reliable and efficient channel for transferring accepted client connections from the master to worker processes. This mechanism enables workers to handle real TCP client sockets without performing the acceptance step themselves.

2.2.5 HTTP Handling Layer

At the final stage of the workflow, the HTTP handling layer processes client requests and generates responses. This component interprets HTTP messages, and constructs appropriate HTTP responses.

2.3. Workflow

The server processes client requests through a clear and structured workflow, from connection acceptance to response delivery. This section provides a detailed description of the data flow, illustrating how client requests traverse the system and how different components interact to handle them efficiently.

Step 1: Client Connection

A client initiates a TCP connection to the server. The master process listens for incoming connections and accepts them as they arrive. At this stage, the connection is represented as a file descriptor that will be passed to a worker process for further handling.

Step 2: Work Dispatch

Once a connection is accepted, the master prepares it for processing by a worker. The connection information is transferred through the Unix Domain Socket (UDS) and the worker receives the connection while ensuring synchronization and preventing race conditions.

Step 3: Worker Processing

The worker enqueues the task into its internal thread pool. Each worker thread can then pick up a request from the queue independently, enabling multiple requests to be processed concurrently within the same worker.

Step 4: Request Handling

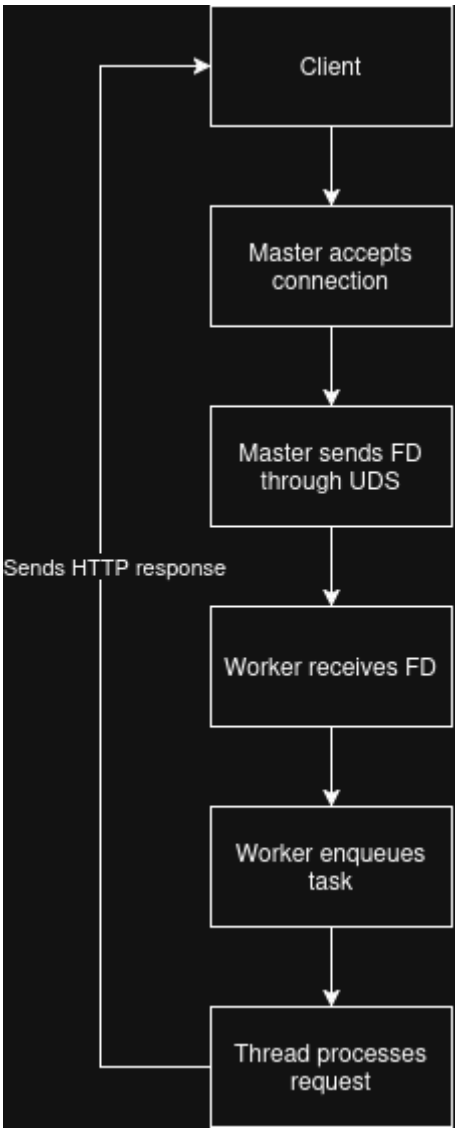
The selected thread reads the HTTP request from the client connection, interprets it, and generates an appropriate response. The HTTP handling logic is encapsulated within the thread, which ensures that request parsing, resource access, and response construction occur independently of other threads and processes.

Step 5: Response Delivery

After processing the request, the thread sends the generated HTTP response back to the client through the same connection. Once the response is sent, the connection is maintained (HTTP Keep-Alive), for at least 5 more seconds of no requests detected, until it is eventually closed.

Step 6: Completion and Cleanup

After the response is delivered, the thread marks the task as complete and becomes available for the next request in the queue. The worker continues processing other pending requests, and the master remains available to accept new client connections, maintaining continuous service availability.



2.4. Architecture Diagram

